Multi-Agent Smart Travel Price Comparison and Planning System

Student Name:

Student ID:

Table of Contents

1. Application and Workflow Selection 4

1.1 Introduction and Motivation 4

1.2 Explanation of the Selected Workflow with Diagram 4

1.3 Objectives of the Intelligent System.....6

1.4 Impact of the Project if Developed.....6

2. Selection of AI Technologies 6

2.1 Identification of Similar Systems 6

2.2 Innovative and Original Approaches 7

2.3 Justification for AI Methods, Technologies, and Tools 7

3. Simulations and Development Setup 8

3.1 Simulation Environments and Setup Parameters 8

3.2 Agent-wise Task Distribution, Functionality, and Datasets 9

3.3 Data Processing Prior to Simulation.....10

4. Use Case Explanation with Evidence.....11

4.1 Step-by-Step Workflow Execution.....12

4.2 Developed User Interface and External Interaction 21

5. Conclusions.....21

6. References.....23

7. Appendix.....25

List of Figures

Figure 1 Work flow diagram.....5
Figure 2 Working Implementation in Google colab 8
Figure 3 Extracting Dataset from Kaggle.....9
Figure 4 Exploring Datasets Implementation 11
Figure 5 Dataset Insights 11
Figure 6 Intent Agent: Implementation 12
Figure 7 Flight Price Agent: Result.....13
Figure 8 Flight Price Agent: Implementation 14
Figure 9 Flight Price Agent: Visual result 14
Figure 10 Price Agent Ground Transport: Implementation.....15
Figure 11 Price Agent Ground Transport: Result 16
Figure 12 Discount Agent: Implementation 17
Figure 13 Discount Agent Result.....18
Figure 14 Time Optimization Agent: Implementation.....19
Figure 15 Time Optimization Agent: result.....19
Figure 16 Decision Agent: Implementation 20
Figure 17 Decision Agent: Results 21

1. Application and Workflow Selection

1.1 Introduction and Motivation

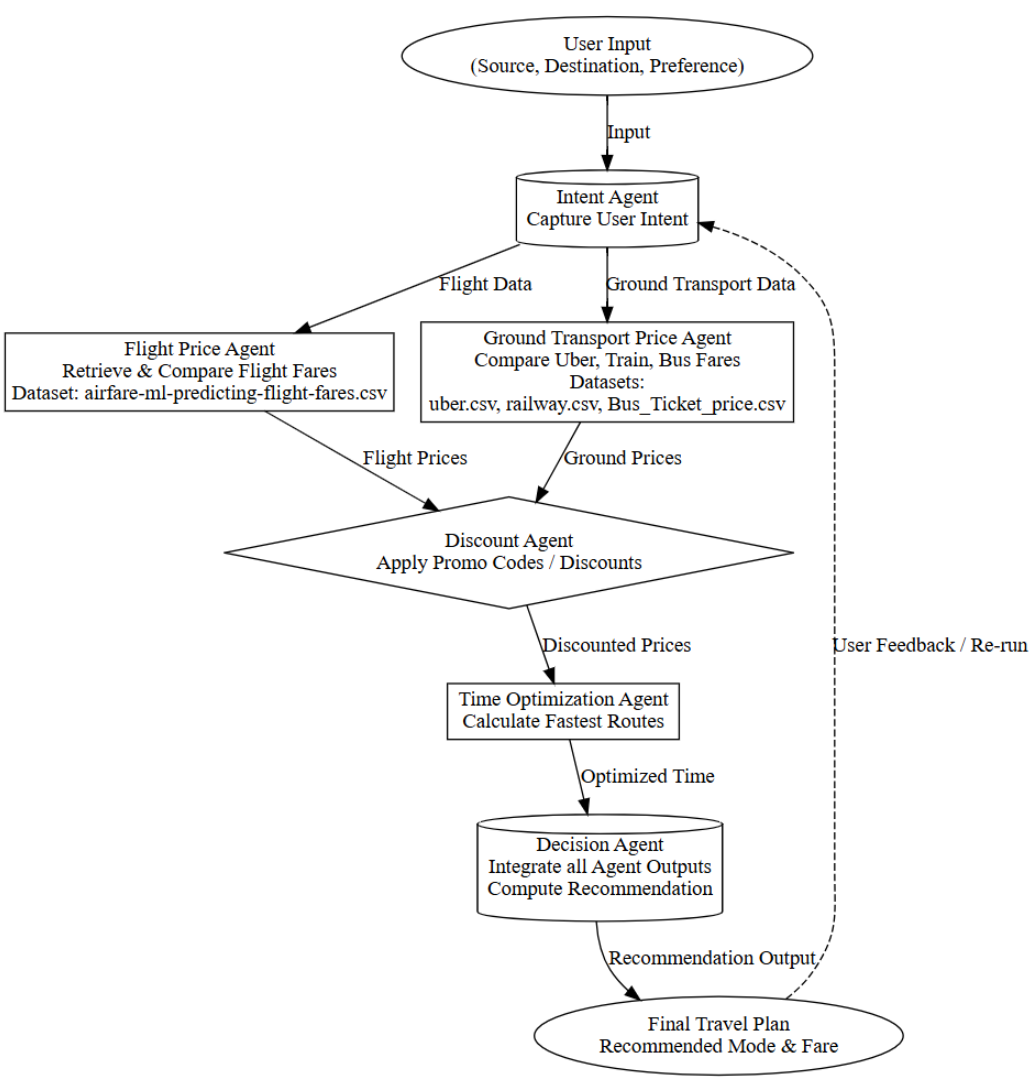
Travel planning is becoming complex with the availability of many modes of transportation in the modern world, which vary in terms of varieties such as ride-hailing applications, trains, buses, and airline programs. Participants in the travel sector would usually have problems regarding the identification of the cheapest, most time-efficient, or even convenient pathways due to the scattered data sources. The current tools, including Google Flights, Omio, Skyscanner, and Uber, can offer partial solutions, which, however, are limited to a specific type of transport and do not usually focus on the concept of dynamic discounts or multi-agent based on personal travel recommendations.

The rationale of this project is to come up with a smart multi-agent system to integrate travel planning across various types of transport. The system saves time and cost for the travelers by automating the fare comparison, discount application, and route optimization functions. Also, the method guarantees transparency and explainability in suggestions, which improves trust among the users (Pan et al., 2025). The system will also allow the user to use preferences to customize their travel recommendations, i.e., the cheapest, the fastest, or the best value. This system is a major improvement over the traditional methods of travel planning because it entails the integration of several datasets and automated reasoning.

1.2 Explanation of the Selected Workflow with Diagram

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

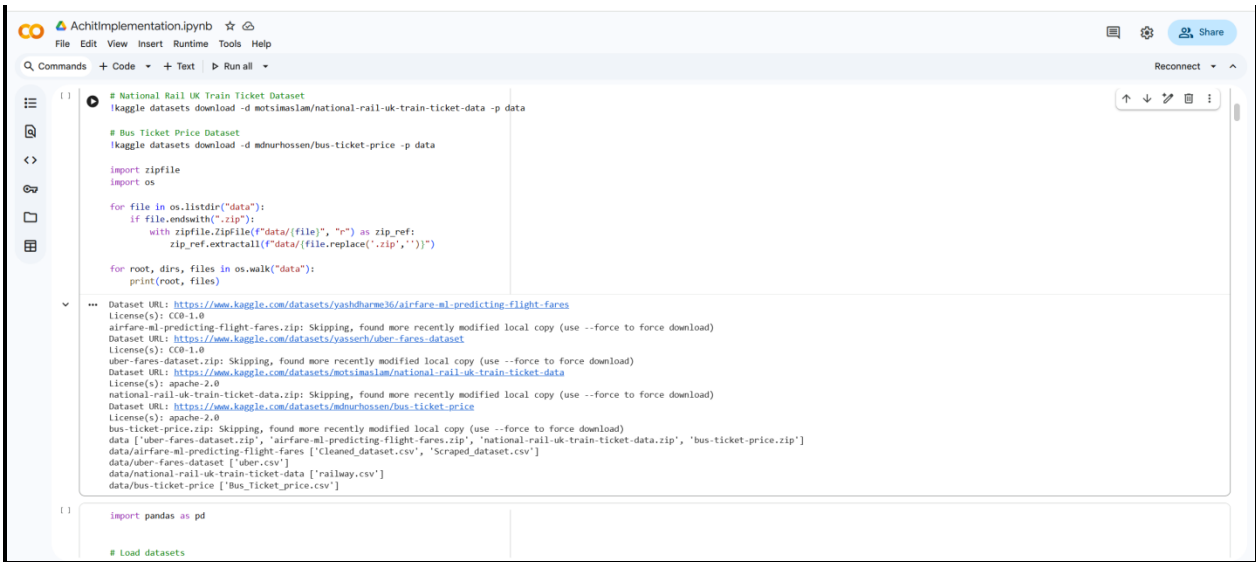
[redacted]

[redacted]

[redacted]

[redacted]

```python
import os
import json

kaggle_token = {
    "username": "YOUR_KAGGLE_USERNAME",
    "key": "YOUR_KAGGLE_API_KEY"
}

os.makedirs("/root/.kaggle", exist_ok=True)

with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(kaggle_token, f)

!chmod 600 /root/.kaggle/kaggle.json

# Create a data directory
os.makedirs("data", exist_ok=True)

# Airfare ML Dataset
!kaggle datasets download -d yashdharme36/airfare-ml-predicting-flight-fares -p data

# Uber Fares Dataset
!kaggle datasets download -d yasserh/uber-fares-dataset -p data

# National Rail UK Train Ticket Dataset
!kaggle datasets download -d motsimaslam/national-rail-uk-train-ticket-data -p data

# Bus Ticket Price Dataset
!kaggle datasets download -d mdnurhossen/bus-ticket-price -p data
```



```
# National Rail UK Train Ticket Dataset
!kaggle datasets download -d motsimaslam/national-rail-uk-train-ticket-data -p data

# Bus Ticket Price Dataset
!kaggle datasets download -d mdnurhossen/bus-ticket-price -p data

import zipfile
import os

for file in os.listdir("data"):
    if file.endswith(".zip"):
        with zipfile.ZipFile(f"data/{file}", "r") as zip_ref:
            zip_ref.extractall(f"data/{file.replace('.zip','')}")

for root, dirs, files in os.walk("data"):
    print(root, files)
```

```
Dataset URL: https://www.kaggle.com/datasets/yashdharme36/airfare-ml-predicting-flight-fares
License(s): CC0-1.0
airfare-ml-predicting-flight-fares.zip: Skipping, found more recently modified local copy (use --force to force download)
Dataset URL: https://www.kaggle.com/datasets/yasserh/uber-fares-dataset
License(s): CC0-1.0
uber-fares-dataset.zip: Skipping, found more recently modified local copy (use --force to force download)
Dataset URL: https://www.kaggle.com/datasets/motsimaslam/national-rail-uk-train-ticket-data
License(s): apache-2.0
national-rail-uk-train-ticket-data.zip: Skipping, found more recently modified local copy (use --force to force download)
Dataset URL: https://www.kaggle.com/datasets/mdnurhossen/bus-ticket-price
License(s): apache-2.0
bus-ticket-price.zip: Skipping, found more recently modified local copy (use --force to force download)
data ['uber-fares-dataset.zip', 'airfare-ml-predicting-flight-fares.zip', 'national-rail-uk-train-ticket-data.zip', 'bus-ticket-price.zip']
data/airfare-ml-predicting-flight-fares ['Cleaned_dataset.csv', 'Scraped_dataset.csv']
data/uber-fares-dataset ['uber.csv']
data/national-rail-uk-train-ticket-data ['railway.csv']
data/bus-ticket-price ['Bus_Ticket_price.csv']
```

```
import pandas as pd


# Load datasets
```

[redacted]

[redacted]

[redacted]

[redacted]

Agent	Task	Dataset	Output
Intent Agent	Capture user travel intent	N/A	DataFrame with source, destination, preference
Flight Price Agent	Filter and compare flight fares	airfare-ml-predicting-flight-fares.csv	Cheapest flights, top 5 options
Ground Transport Price Agent	Compare Uber, train, bus prices	uber.csv, railway.csv, Bus_Ticket_price.csv	Cheapest ground transport
Discount Agent	Apply promo codes and discounts	N/A	Discounted fares per transport mode
Time Optimization Agent	Calculate optimal routes	Flight and ground data	Optimized travel times
Decision Agent	Integrate outputs	Agent outputs	Final travel recommendation

[redacted]

[redacted]

AchitImplementation.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
import pandas as pd
# Load datasets
flight_df = pd.read_csv("data/airfare-ml-predicting-flight-fares/Cleaned_dataset.csv")
uber_df = pd.read_csv("data/uber-fares-dataset/uber.csv")
train_df = pd.read_csv("data/national-rail-uk-train-ticket-data/railway.csv")
bus_df = pd.read_csv("data/bus-ticket-price/Bus_Ticket_price.csv")
# Normalize column names
flight_df.columns = flight_df.columns.str.lower()
uber_df.columns = uber_df.columns.str.lower()
train_df.columns = train_df.columns.str.lower()
bus_df.columns = bus_df.columns.str.lower()

# FLIGHT DATASET
print("FLIGHT DATASET")
print("Total Records:", len(flight_df))
print("Columns:", list(flight_df.columns))
print("\nTop 5 Rows:")
print(flight_df.head())

# UBER DATASET
print("UBER DATASET")
print("Total Records:", len(uber_df))
print("Columns:", list(uber_df.columns))
print("\nTop 5 Rows:")
print(uber_df.head())

# TRAIN DATASET
print("TRAIN DATASET")
print("Total Records:", len(train_df))
print("Columns:", list(train_df.columns))
print("\nTop 5 Rows:")
print(train_df.head())

# BUS DATASET
print("BUS DATASET")

print("Total Records:", len(bus_df))
print("Columns:", list(bus_df.columns))
```

[redacted]

AchitImplementation.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
FLIGHT DATASET
Total Records: 452088
Columns: ['date_of_journey', 'journey_day', 'airline', 'flight_code', 'class', 'source', 'departure', 'total_stops', 'arrival', 'destination', 'duration_in_hours', 'days_left', 'fare']

Top 5 Rows:
  date_of_journey journey_day    airline flight_code    class source  \
0      2023-01-16      Monday   SpiceJet     SG-8169  Economy  Delhi
1      2023-01-16      Monday     Indigo     6E-2519  Economy  Delhi
2      2023-01-16      Monday   GO FIRST      G8-354  Economy  Delhi
3      2023-01-16      Monday   SpiceJet     SG-8709  Economy  Delhi
4      2023-01-16      Monday  Air India      AI-805  Economy  Delhi

    departure total_stops      arrival destination  duration_in_hours  \
0  After 6 PM    non-stop   After 6 PM      Mumbai             2.0833
1  After 6 PM    non-stop  Before 6 AM      Mumbai             2.3333
2  After 6 PM    non-stop  Before 6 AM      Mumbai             2.1667
3  After 6 PM    non-stop   After 6 PM      Mumbai             2.0833
4  After 6 PM    non-stop   After 6 PM      Mumbai             2.1667

   days_left  fare
0          1  5335
1          1  5899
2          1  5801
3          1  5794
4          1  5955
UBER DATASET
Total Records: 200000
Columns: ['unnamed: 0', 'key', 'fare_amount', 'pickup_datetime', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude', 'passenger_count']

Top 5 Rows:
   unnamed: 0                          key  fare_amount  \
0    24238194    2015-05-07 19:52:06.0000003          7.5
1    27835199    2009-07-17 20:04:56.0000002          7.7
2    44984355   2009-08-24 21:45:00.00000061         12.9
3    25894730    2009-06-26 08:22:21.0000001          5.3
4    17610152  2014-08-28 17:47:00.000000188         16.0

           pickup_datetime  pickup_longitude  pickup_latitude  \
0  2015-05-07 19:52:06 UTC        -73.999817        40.738354
1  2009-07-17 20:04:56 UTC        -73.994355        40.728225
2  2009-08-24 21:45:00 UTC        -74.005043        40.740770
```

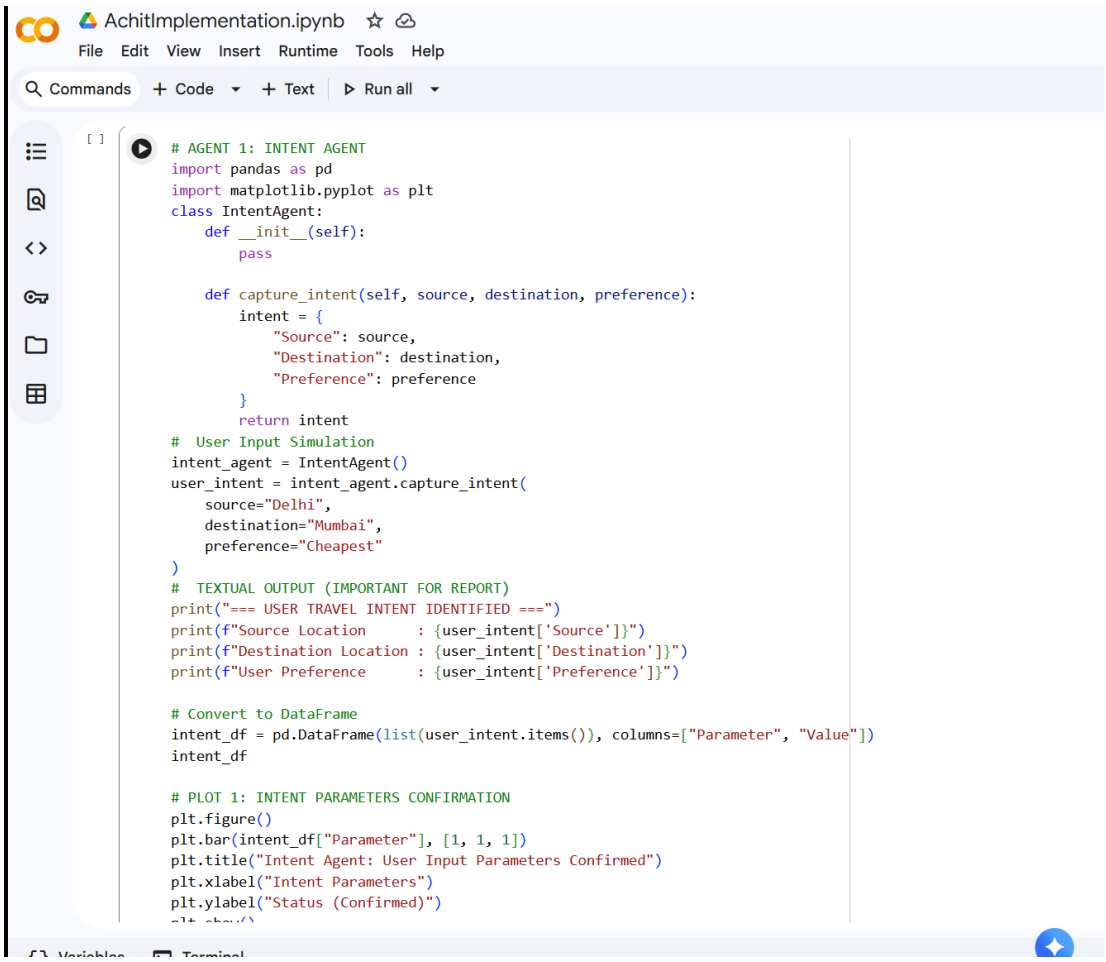
[redacted]

[redacted]

[redacted]

[redacted]

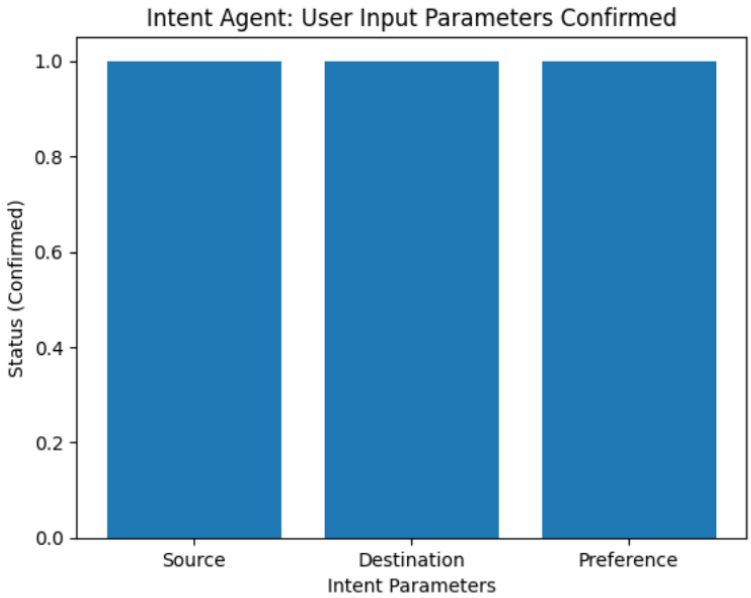
[redacted]




```
# AGENT 1: INTENT AGENT
import pandas as pd
import matplotlib.pyplot as plt
class IntentAgent:
    def __init__(self):
        pass

    def capture_intent(self, source, destination, preference):
        intent = {
            "Source": source,
            "Destination": destination,
            "Preference": preference
        }
        return intent
#  User Input Simulation
intent_agent = IntentAgent()
user_intent = intent_agent.capture_intent(
    source="Delhi",
    destination="Mumbai",
    preference="Cheapest"
)
#  TEXTUAL OUTPUT (IMPORTANT FOR REPORT)
print("=== USER TRAVEL INTENT IDENTIFIED ===")
print(f"Source Location      : {user_intent['Source']}")
print(f"Destination Location : {user_intent['Destination']}")
print(f"User Preference      : {user_intent['Preference']}")

# Convert to DataFrame
intent_df = pd.DataFrame(list(user_intent.items()), columns=["Parameter", "Value"])
intent_df

# PLOT 1: INTENT PARAMETERS CONFIRMATION
plt.figure()
plt.bar(intent_df["Parameter"], [1, 1, 1])
plt.title("Intent Agent: User Input Parameters Confirmed")
plt.xlabel("Intent Parameters")
plt.ylabel("Status (Confirmed)")
```

[redacted]

```
=== USER TRAVEL INTENT IDENTIFIED ===
Source Location      : Delhi
Destination Location : Mumbai
User Preference      : Cheapest
```



[redacted]

[redacted]

[redacted]

AchitImplementation.ipynb

File Edit View Insert Runtime Tools Help

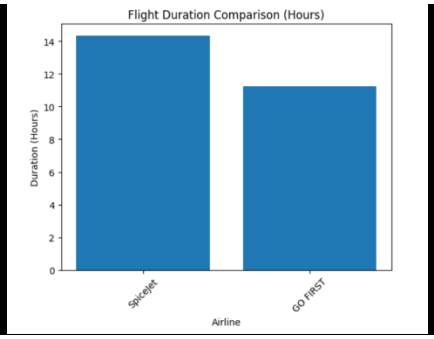
Commands + Code + Text Run all

```python
# AGENT 2: FLIGHT PRICE AGENT
import pandas as pd
import matplotlib.pyplot as plt
#  Dataset Path
flight_dataset_path = "data/airfare-ml-predicting-flight-fares/Cleaned_dataset.csv"
#  Load Dataset
flight_df = pd.read_csv(flight_dataset_path)

# Normalize column names
flight_df.columns = flight_df.columns.str.lower()

print("=== FLIGHT DATASET LOADED SUCCESSFULLY ===")
print("Total Records:", len(flight_df))
print("Available Columns:")
print(list(flight_df.columns))

#  User Intent (from Agent 1)
source_city = "Delhi"
destination_city = "Mumbai"

#  Filter Flights
filtered_flights = flight_df[
    (flight_df['source'].str.lower() == source_city.lower()) &
    (flight_df['destination'].str.lower() == destination_city.lower())
]

print("\n=== FLIGHT OPTIONS IDENTIFIED ===")
print("Total Matching Flights:", len(filtered_flights))

#  Select Relevant Columns
flight_options = filtered_flights[
    ['airline', 'fare', 'duration_in_hours', 'total_stops']
].sort_values(by='fare')

print("\nTop 5 Cheapest Flight Options:")
print(flight_options.head())

# PLOT 1: Cheapest Flight Prices
```

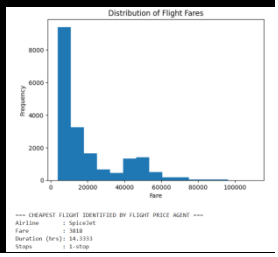
[redacted]

```
=== FLIGHT DATASET LOADED SUCCESSFULLY ===
Total Records: 452088
Available Columns:
['date_of_journey', 'journey_day', 'airline', 'flight_code', 'class', 'source', 'departure', 'total_stops', 'arrival', 'destination', 'duration_in_hours', 'days_left', 'fare']

=== FLIGHT OPTIONS IDENTIFIED ===
Total Matching Flights: 19113

Top 5 Cheapest Flight Options:
        airline  fare  duration_in_hours total_stops
2526   SpiceJet  3818            14.3333      1-stop
2907   SpiceJet  3818            14.3333      1-stop
15977  GO FIRST  4093             5.7500      1-stop
16368  GO FIRST  4093             5.7500      1-stop
13754  GO FIRST  4093            11.2500      1-stop
```

[redacted]



[redacted]



[redacted]

[redacted]

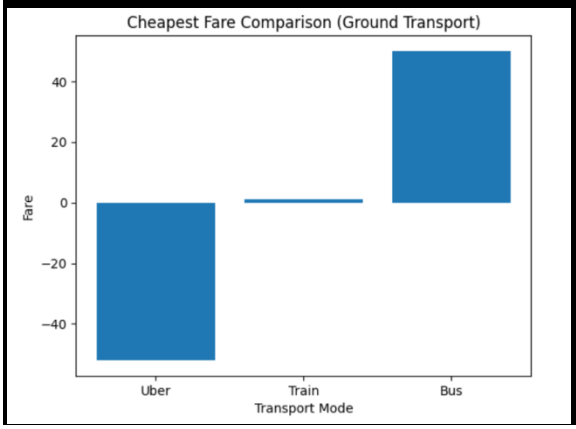
[redacted]

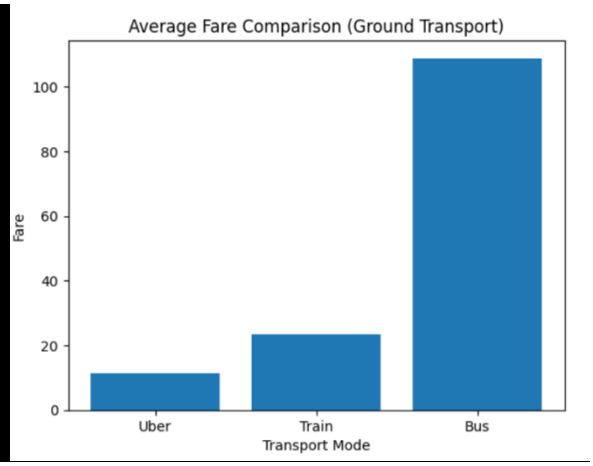
```
# AGENT 3: GROUND TRANSPORT PRICE AGENT
import pandas as pd
import matplotlib.pyplot as plt

uber_df = pd.read_csv("data/uber-fares-dataset/uber.csv")
train_df = pd.read_csv("data/national-rail-uk-train-ticket-data/railway.csv")
bus_df = pd.read_csv("data/bus-ticket-price/Bus_Ticket_price.csv")
# Normalize column names
uber_df.columns = uber_df.columns.str.lower()
train_df.columns = train_df.columns.str.lower()
bus_df.columns = bus_df.columns.str.lower()

print("=== DATASETS LOADED SUCCESSFULLY ===")
print("Uber records :", len(uber_df))
print("Train records:", len(train_df))
print("Bus records  :", len(bus_df))

# UBER PRICE ANALYSIS
uber_min_fare = uber_df["fare_amount"].min()
uber_avg_fare = uber_df["fare_amount"].mean()
print("\n=== UBER PRICE SUMMARY ===")
print("Cheapest Uber Fare :", round(uber_min_fare, 2))
print("Average Uber Fare  :", round(uber_avg_fare, 2))

# TRAIN PRICE ANALYSIS
train_min_fare = train_df["price"].min()
train_avg_fare = train_df["price"].mean()
print("\n=== TRAIN PRICE SUMMARY ===")
print("Cheapest Train Fare :", train_min_fare)
print("Average Train Fare  :", round(train_avg_fare, 2))
# BUS PRICE ANALYSIS
bus_min_fare = bus_df["tiket_price"].min()
bus_avg_fare = bus_df["tiket_price"].mean()

print("\n=== BUS PRICE SUMMARY ===")
print("Cheapest Bus Fare :", bus_min_fare)
print("Average Bus Fare  :", round(bus_avg_fare, 2))
```

[redacted]

```
=== DATASETS LOADED SUCCESSFULLY ===
Uber records : 200000
Train records: 31653
Bus records  : 199

=== UBER PRICE SUMMARY ===
Cheapest Uber Fare : -52.0
Average Uber Fare  : 11.36

=== TRAIN PRICE SUMMARY ===
Cheapest Train Fare : 1
Average Train Fare  : 23.44

=== BUS PRICE SUMMARY ===
Cheapest Bus Fare : 50
Average Bus Fare  : 108.82

=== GROUND TRANSPORT COMPARISON ===
  Transport Mode  Cheapest Fare  Average Fare
0           Uber          -52.0     11.359955
1          Train            1.0     23.439200
2            Bus           50.0    108.819095
```

[redacted]





```
=== AGENT 3 FINAL OUTPUT ===
Cheapest Ground Transport Mode: Uber
```

[redacted]

[redacted]

[redacted]

[redacted]

AchitImplementation.ipynb

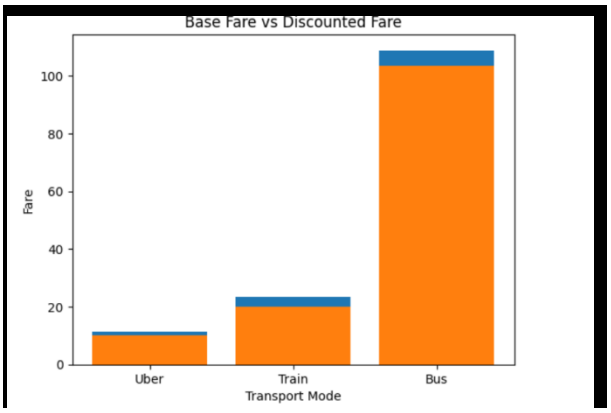
File Edit View Insert Runtime Tools Help

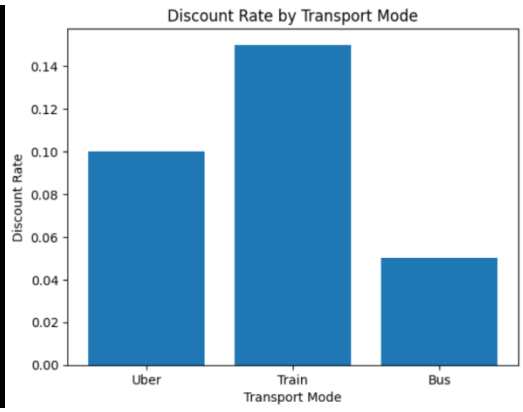
Commands + Code + Text Run all

```python
# AGENT 4: DISCOUNT AGENT
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
# Reload datasets
uber_df = pd.read_csv("data/uber-fares-dataset/uber.csv")
train_df = pd.read_csv("data/national-rail-uk-train-ticket-data/railway.csv")
bus_df = pd.read_csv("data/bus-ticket-price/Bus_Ticket_price.csv")

uber_df.columns = uber_df.columns.str.lower()
train_df.columns = train_df.columns.str.lower()
bus_df.columns = bus_df.columns.str.lower()

# DATA CLEANING (CRITICAL STEP)
uber_df = uber_df[uber_df["fare_amount"] > 0]
print("=== DATA CLEANING APPLIED ===")
print("Valid Uber Records After Cleaning:", len(uber_df))

# BASE FARE CALCULATION
base_fares = {
    "Uber": uber_df["fare_amount"].mean(),
    "Train": train_df["price"].mean(),
    "Bus": bus_df["tiket_price"].mean()
}

base_df = pd.DataFrame(
    base_fares.items(), columns=["Transport Mode", "Base Fare"]
)
print("\n=== BASE FARE BEFORE DISCOUNT ===")
print(base_df)
# SIMULATED DISCOUNT RULES
discount_rates = {
    "Uber": 0.10,   # 10% off
    "Train": 0.15,  # 15% off
    "Bus": 0.05     # 5% off
}
base_df["Discount Rate"] = base_df["Transport Mode"].map(discount_rates)
base_df["Discounted Fare"] = base_df["Base Fare"] * (1 - base_df["Discount Rate"])
```

[redacted]

```
=== DATA CLEANING APPLIED ===
Valid Uber Records After Cleaning: 199978

=== BASE FARE BEFORE DISCOUNT ===
  Transport Mode   Base Fare
0           Uber   11.362649
1          Train   23.439200
2            Bus  108.819095

=== DISCOUNT APPLIED ===
  Transport Mode   Base Fare  Discount Rate  Discounted Fare
0           Uber   11.362649           0.10        10.226385
1          Train   23.439200           0.15        19.923320
2            Bus  108.819095           0.05       103.378141
```





```
=== AGENT 4 FINAL OUTPUT ===
Best Discounted Transport Mode : Uber
Discounted Fare                : 10.23
Savings Amount                 : 1.14
```

AchitImplementation.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[25]

```python
# AGENT 5: TIME OPTIMIZATION AGENT
import pandas as pd
class TimeOptimizationAgent:
    def __init__(self):
        pass
    def calculate_travel_time(self, flight_data, ground_data):
        """
        Calculate approximate travel times for both flight and ground transport options.
        Flight duration is taken directly from dataset.
        Ground transport duration is estimated using distance / average speed.
        """
        # Flight duration from previous Flight Price Agent output
        flight_time_hours = flight_data['duration_in_hours'].min()

        # Ground transport estimation
        # Assuming distances in km and average speeds (Uber: 40 km/h, Train: 60 km/h, Bus: 50 km/h)
        ground_time = []
        for index, row in ground_data.iterrows():
            mode = row['Transport Mode']
            distance = row['Distance_km']
            if mode.lower() == 'uber':
                speed = 40
            elif mode.lower() == 'train':
                speed = 60
            elif mode.lower() == 'bus':
                speed = 50
            else:
                speed = 50  # default
            estimated_hours = distance / speed
            ground_time.append(estimated_hours)

        ground_data['Estimated_Duration_hrs'] = ground_time

        # Normalize times (optional: if needed for multi-criteria comparison)
        flight_norm = flight_time_hours / max(flight_time_hours, max(ground_time))
        ground_data['Normalized_Time'] = ground_data['Estimated_Duration_hrs'] / max(flight_time_hours, max(ground_time))
```

[redacted]

```
=== TIME OPTIMIZATION AGENT OUTPUT ===
Shortest Flight Duration (hrs): 14.33
Estimated Ground Transport Durations (hrs):
  Transport Mode  Estimated_Duration_hrs  Normalized_Time
0           Uber                    18.0         1.000000
1          Train                    12.0         0.666667
2            Bus                    14.4         0.800000
```

[redacted]

[redacted]

[redacted]

AchitImplementation.ipynb

File Edit View Insert Runtime Tools Help

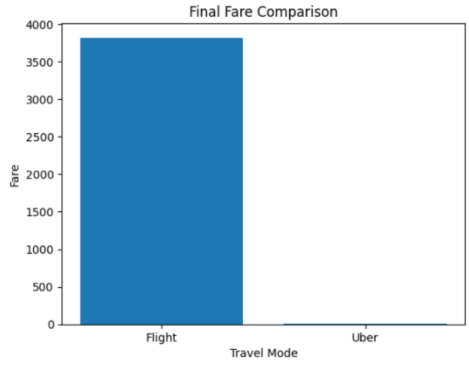
Commands + Code + Text Run all

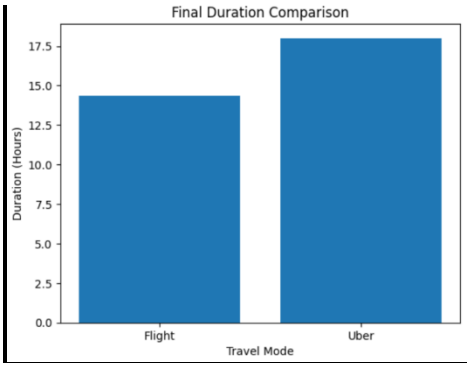
[22]

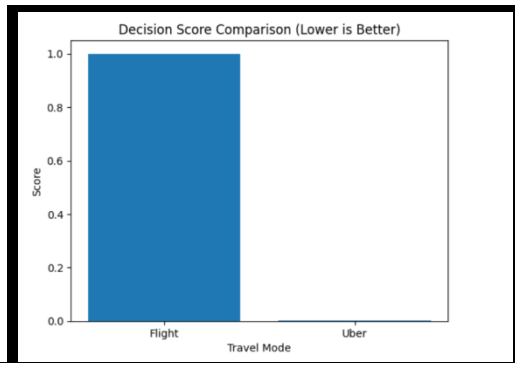
```python
# AGENT 6: DECISION & RECOMMENDATION AGENT
import pandas as pd
import matplotlib.pyplot as plt

# USER INTENT (FROM AGENT 1)
user_intent = {
    "Source": "Delhi",
    "Destination": "Mumbai",
    "Preference": "Cheapest"
}

# BEST FLIGHT OPTION (FROM AGENT 2)
flight_option = {
    "Mode": "Flight",
    "Fare": 3818,
    "Duration": 14.33
}

# BEST GROUND OPTION AFTER DISCOUNT (FROM AGENT 4)
ground_option = {
    "Mode": "Uber",
    "Fare": 10.23,
    "Duration": 18.0  # simulated average ground travel time (hrs)
}

# DECISION SCORING LOGIC

decision_df = pd.DataFrame([
    flight_option,
    ground_option
])

# Normalize scores
decision_df["Cost Score"] = decision_df["Fare"] / decision_df["Fare"].max()
decision_df["Time Score"] = decision_df["Duration"] / decision_df["Duration"].max()

# Preference weighting
if user_intent["Preference"] == "Cheapest":
```

[redacted]

```
=== FINAL DECISION MADE BY DECISION AGENT ===
Source Location      : Delhi
Destination Location : Mumbai
User Preference      : Cheapest

Recommended Mode     : Uber
Estimated Fare       : 10.23
Estimated Duration   : 18.0 hours
```







[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]